# Laboratorio 04

File, processi ed exec in C: copy_input, cerca, anteprima

Questo documento spiega i file della cartella **Lab04-soluzioni**: struttura dei programmi, significato di ogni parte, cosa fanno e come rifare gli esercizi da zero in vista dell'esame.

| File | Esercizio | Idea in una riga |
|---|---|---|
| Lab04-es01-copy_input.c | Es. 1 | Legge righe da tastiera e le scrive su un file finché l'utente digita "fine". |
| Lab04-es02-cerca.c | Es. 2 | Un figlio per ogni stringa: ognuno scrive un log e lancia *grep -c* sul file. Esecuzione **concorrente**. |
| Lab04-es03-anteprima.c | Es. 3 | Chiede nomi di file e per ciascuno lancia *head -n* in un figlio. Esecuzione **sequenziale**. |

## 1. Concetti comuni

| Concetto | Significato |
|---|---|
| File descriptor | Intero che identifica un file aperto. 0 = stdin, 1 = stdout, 2 = stderr; *open* restituisce il primo libero (di solito 3). Valore negativo = errore. |
| `open(path, flag, mode)` | Apre o crea un file. Flag usati: *O_RDONLY*, *O_WRONLY*, *O_CREAT* (crea se manca), *O_TRUNC* (svuota se esiste), *O_APPEND* (scrive sempre in coda). *mode* (es. 0644) serve solo con O_CREAT. |
| Permessi 0644 | rw-r--r--: lettura/scrittura al proprietario, sola lettura a gruppo e altri. Il valore effettivo è ridotto dalla *umask*. |
| `write(fd, buf, n)` | Scrive n byte dal buffer sul file. Restituisce i byte scritti o -1. |
| `close(fd)` | Chiude il file. Dimenticarlo causa un *resource leak*. |
| `errno / perror()` | *errno* contiene il codice dell'ultimo errore (es. *ENOENT* = file inesistente); *perror("msg")* stampa il messaggio leggibile. |
| `fork()` | Duplica il processo: 0 nel figlio, PID del figlio nel padre, negativo se errore. |
| `execlp(file, arg0, arg1, ..., NULL)` | **Sostituisce** il programma del processo con un altro. La *p* indica che il comando viene cercato nel PATH. Il primo argomento dopo il nome è *argv[0]* (si ripete il nome del comando) e la lista termina con *(char *)0*. **Non ritorna** se ha successo. |
| `wait(&status;)` | Il padre attende la terminazione di un figlio. |

**Schema fork + exec + wait:** è il modo standard per lanciare un altro programma e continuare a lavorare. Il figlio fa *exec*, il padre fa *wait*. Le righe dopo *execlp* nel figlio vengono eseguite **solo se exec fallisce**, quindi lì ci sono *perror* ed *exit*.

## 2. Esercizio 1: copy_input

**Uso:** `./copy_input filename`

**Cosa fa.** Crea (o svuota) il file indicato, poi chiede stringhe da tastiera e le scrive nel file una per riga, finché l'utente digita *fine*.

### Struttura

1. Controllo argomenti: *argc != 2* stampa l'uso su *stderr* ed esce con codice 1.
2. **open** con *O_WRONLY* | *O_TRUNC* | *O_CREAT* e permessi 0644. Se *fd* < *0* errore e *exit(2)*.
3. **fgets(buff, 1024, stdin)** legge una riga **compreso il carattere \n**.
4. Ciclo `while (strcmp(buff, "fine\n") != 0)`: scrive la riga con `write(fd, buff, strlen(buff))` e ne legge un'altra. Si confronta con "fine\n" proprio perché fgets conserva il newline.
5. **close(fd)** e uscita.

### Punti da sapere

- La riga "fine" **non** viene scritta nel file.
- Si scrive *strlen(buff)* byte e non *sizeof(buff)*, altrimenti si copierebbe anche la spazzatura dopo il terminatore.
- Alternativa: *scanf("%s")* legge una parola senza spazi e senza newline; in tal caso si confronta con "fine".
- Difetto: se l'input termina (CTRL-D) *fgets* restituisce NULL e *buff* non cambia, quindi il ciclo potrebbe non finire. Una versione robusta controlla il valore di ritorno di fgets.
- Una riga più lunga di 1023 caratteri viene spezzata da fgets in più letture.

## 3. Esercizio 2: cerca

**Uso:** <font name='Courier'>./cerca <nomefile> <stringa1> ... <stringaN></font>

**Cosa fa.** Per ogni stringa il padre crea un figlio. Ogni figlio registra la propria operazione nel file *conteggio.txt* e poi lancia `grep -c stringa nomefile`, che stampa **quante righe** del file contengono la stringa. I figli lavorano **in modo concorrente**.

### Struttura

1. Controllo: servono almeno 3 argomenti (*argc* < *3* = errore). *argv[1]* è il file, da *argv[2]* in poi le stringhe.
2. Il padre crea/svuota *conteggio.txt* con *O_CREAT* | *O_TRUNC* | *O_WRONLY* e lo **chiude subito**: serve solo a partire da un file vuoto.
3. Ciclo *for (i = 2; i < argc; i++)* con una *fork* per iterazione.
4. **Figlio**: apre *conteggio.txt* con *O_WRONLY* | *O_APPEND*, scrive la riga di log "nomefile stringa" (costruita con *sprintf*), chiude il file, stampa un'intestazione ed esegue *execlp("grep", "grep", "-c", stringa, nomefile, (char *)0)*. Se exec fallisce: *perror* ed *exit(4)*.
5. **Padre**: finito il ciclo di fork, fa *wait* tante volte quanti sono i figli (*argc - 2*).

### Punti chiave per l'esame

- **Concorrenza**: tutti i figli partono insieme, quindi l'**ordine** delle righe stampate e del log nel file non è deterministico.
- **O_APPEND** è ciò che rende sicuro il log: ogni *write* posiziona il cursore in coda e scrive in un'unica operazione, quindi i figli non si sovrascrivono. Senza O_APPEND (con un semplice *O_WRONLY*) ogni figlio scriverebbe dall'inizio del file e si perderebbero righe.
- Il figlio **non rientra mai nel for**: o fa exec o termina con exit. Senza questa garanzia ogni figlio creerebbe altri processi.
- Il file viene aperto **dal figlio** perché ogni processo deve avere il proprio descrittore. L'exec poi sostituisce il programma: ecco perché prima va fatto il log.
- *grep -c* conta le **righe** che contengono la stringa, non le occorrenze totali (una riga con due occorrenze vale 1).

- Il numero di *wait* deve coincidere con il numero di figli, altrimenti restano processi zombie.

### Difetti noti

- Il valore di ritorno della *open* nel figlio non viene controllato.
- Il testo stampato con *printf* prima di *execlp* termina con \n e finisce a video, ma se lo standard output fosse rediretto a file o pipe il buffer di stdio potrebbe andare perso, perché *exec* non lo svuota. In quel caso serve *fflush(stdout)*.

## 4. Esercizio 3: anteprima

**Uso:** `./anteprima <num>`

**Cosa fa.** Chiede in ciclo il nome di un file. Se il file esiste mostra le prime *num* righe lanciando `head -n num nomefile` in un figlio. Il ciclo termina digitando *fine*. L'esecuzione è **sequenziale**.

### Struttura

1. Controllo argomenti: serve esattamente un parametro, convertito con *atoi*; se il risultato è <= 0 errore con *exit(2)*.
2. Lettura del nome con *scanf("%s", nome)* (*nome* è un array di DIM = 50 caratteri).
3. **while (strcmp(nome, "fine") != 0)**: per ogni nome si controlla se il file esiste provando ad aprirlo con *open(..., O_RDONLY)*.
4. Se *open* fallisce con *errno* == *ENOENT* si stampa "il file non esiste" e si chiede il nome successivo; con qualunque altro errore (es. permessi) *perror* ed *exit(3)*.
5. Se *open* riesce si **chiude subito il descrittore** (altrimenti resource leak): serviva solo per verificare l'esistenza.
6. *fork*: il **figlio** stampa l'intestazione ed esegue *execlp("head", "head", "-n", argv[1], nome, (char *)0)*.
7. Il **padre** fa *wait* subito dopo la fork e solo dopo richiede il nome successivo.

### Punti chiave per l'esame

- **Sequenziale vs concorrente**: qui il *wait* è dentro il ciclo, subito dopo la fork, quindi c'è un solo figlio alla volta. Nell'Es. 2 le fork stanno tutte nel ciclo e i *wait* sono dopo.
- Perché serve la *fork*: se il padre facesse direttamente *exec*, il programma verrebbe sostituito da *head* e non potrebbe più chiedere altri file.
- *ENOENT* distingue "file inesistente" da altri errori: *open* può fallire anche se il file esiste (permessi mancanti).
- Dopo *fork* + *exec* il figlio non ritorna al *while*: o fa exec, o esce con *exit(5)* se exec fallisce.

### Difetti noti

- *scanf("%s")* senza limite di larghezza può causare buffer overflow se il nome supera 49 caratteri; meglio *scanf("%49s", nome)*.
- *atoi* non segnala gli errori di conversione ("abc" dà 0); meglio *strtol*.
- Nel secondo messaggio di richiesta c'è un refuso ("visualizzare 'anteprima").

## 5. Confronto tra Es. 2 ed Es. 3

| | Es. 2 - cerca | Es. 3 - anteprima |
|---|---|---|
| Programma lanciato | grep -c | head -n |
| Quanti figli | uno per ogni stringa, tutti insieme | uno alla volta |

| | Es. 2 - cerca | Es. 3 - anteprima |
|---|---|---|
| Dove sta wait | dopo il ciclo di fork | subito dopo la fork, dentro il ciclo |
| Modello | **concorrente** | **sequenziale** |
| Input | argomenti da riga di comando | nomi letti da tastiera |
| File di supporto | conteggio.txt scritto in append | nessuno (solo controllo di esistenza) |

# 6. Come rifare gli esercizi: metodo

Scheletro per i programmi che lanciano altri comandi:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(int argc, char **argv) {
    int pid, status, i;

    /* 1. controllo argomenti */
    if (argc < 3) { fprintf(stderr, "Uso: ...\n"); exit(1); }

    /* 2. (se serve) preparo i file con open/close */

    /* 3. fork */
    pid = fork();
    if (pid < 0) { perror("fork"); exit(2); }
    if (pid == 0) {
        /* figlio: eventuale lavoro, poi exec */
        execlp("comando", "comando", "arg1", "arg2", (char *)0);
        perror("exec");      /* qui solo se exec fallisce */
        exit(3);
    }

    /* 4. padre: wait (subito = sequenziale, dopo il ciclo = concorrente) */
    wait(&status);
    return 0;
}
```

## Ordine di ragionamento davanti a una traccia d'esame

1. Individua **quale comando di sistema** bisogna lanciare (grep, head, wc, ls...) e con quali argomenti.
2. Decidi se serve **un figlio alla volta** (wait nel ciclo) o **tutti insieme** (wait dopo il ciclo).
3. Elenca i file coinvolti e scegli i flag di *open*: lettura, scrittura, creazione, troncamento o append.
4. Assicurati che il figlio **non possa rientrare nel ciclo** del padre (exec seguito da exit).
5. Controlla i valori di ritorno: *open*, *fork* ed *execlp* (con *perror*).
6. Chiudi sempre i file aperti e fai tanti *wait* quanti sono i figli.

## Compilazione e test

```
gcc -Wall -o copy_input Lab04-es01-copy_input.c
./copy_input out.txt            # scrivi qualche riga, poi: fine
cat out.txt

gcc -Wall -o cerca Lab04-es02-cerca.c
./cerca out.txt ciao mondo      # poi: cat conteggio.txt

gcc -Wall -o anteprima Lab04-es03-anteprima.c
./anteprima 3                   # inserisci nomi di file, poi: fine
```

# 7. Checklist di ripasso

- Differenza tra *fgets* (conserva \n) e *scanf("%s")* (legge una parola).
- Significato dei flag *O_CREAT*, *O_TRUNC*, *O_APPEND* e dei permessi 0644.
- Perché *O_APPEND* rende sicura la scrittura concorrente sullo stesso file.
- Cosa fa *execlp*, perché non ritorna e perché si ripete il nome del comando come primo argomento.
- Perché serve *fork* prima di *exec* quando il padre deve continuare a lavorare.
- Differenza tra esecuzione sequenziale e concorrente in base alla posizione di *wait*.
- Come distinguere "file inesistente" da altri errori di *open* con *errno* ed *ENOENT*.
- Perché si chiude il descrittore dopo un'apertura usata solo per controllare l'esistenza.